

# CLAUDE CODE CHAT HISTORY

## Galaga Repair Lab | Lab 4: VibeCoding

Project: 31\_galaga

Session ID: 80c3398a-5751-42c9-a42d-db96f6ff1101

Session date: 8 October 2026

### Transcript scope

Readable transcript of visible user prompts and Claude responses from the recorded Claude Code session. Internal reasoning blocks and raw tool payloads are not reproduced. The original session export remains available in the Lab-4 folder.

#### 01 USER PROMPT

dir

#### 02 CLAUDE RESPONSE

C:\Users\Lenovo\Downloads\SE lab 4\31\_galaga holds 3 files:

| File       | Size    |
|------------|---------|
| .gitignore | 14 B    |
| game.py    | 9,350 B |
| README.md  | 4,205 B |

This is your Galaga game repo, not the MBS prepayment ML project that Downloads\CLAUDE.md describes. If you want to work on the ML project, tell me where it lives or whether I should set it up in a new folder.

#### 03 USER PROMPT

We are now working on Task 1 from the README: Fix the entry/dive path bug.

Before making changes:

- Inspect game.py, especially the bezier() function and every place where it is used.
- Compare the implementation against the standard cubic Bezier formula:  

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3$$
- Pay particular attention to the exponents of  $u = 1-t$  and  $t$  in the two middle terms.
- Confirm the exact bug before changing anything.

Then:

1. Fix only the Bezier formula bug.
2. Do not implement Tasks 2, 3, or 4 yet.
3. Do not make unrelated refactors or changes.
4. Preserve the existing gameplay and controls.
5. Run/test the game after the fix.
6. Explain what was wrong and what you changed.

Do NOT commit the changes. I will make the Git commit myself.

#### 04 CLAUDE RESPONSE

bezier() is the only place the formula appears, and all three paths (entry, dive, return) go through it at line 73. Applying the one-factor fix:

#### 05 CLAUDE RESPONSE

Now testing: I'll check the fixed formula against a reference, compare the old and new versions on a real entry path, and run the game headless for a few seconds.

#### 06 CLAUDE RESPONSE

I fixed the Task 1 path bug with a one-line change in bezier(). Nothing is committed.

What was wrong (game.py:12-13)

The P2 term was missing a factor of t. The formula should be:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3$$

| Term | Code before the fix      | Correct?              |
|------|--------------------------|-----------------------|
| P0   | $u^3$                    | yes                   |
| P1   | $3 * u * u * t = 3u^2 t$ | yes                   |
| P2   | $3 * u * t = 3ut$        | no, should be $3ut^2$ |

| P3 | t \*\* 3 | yes |

The four weights in a Bezier curve should always add up to 1. With the bug they added up to more than 1 partway through the path (about 1.42 at  $t = 0.25$ ), which pushed enemies outward and caused the overshoot. At  $t = 0$  and  $t = 1$  the faulty term is zero, so the start and end points were still correct. That's why checking the endpoints didn't reveal the bug.

What I changed: I replaced  $3 * u * t * p2[\dots]$  with  $3 * u * t * t * p2[\dots]$  on both the x and y lines. `bezier()` is called in one place (`game.py:73`), and the entry, dive and return-to-formation paths all go through it, so this one change fixes all three. Nothing else in the file changed, and Tasks 2???"4 are still empty.

#### Testing

- Formula check: across 10,000 random curves, the fixed `bezier()` matches the standard formula to within about  $1e-13$ .
- A real entry path, before and after: the start ( $t = 0$ ) and end ( $t = 1$ ) points are identical. In between, the old path was pushed out by up to about 42 px (at  $t = 0.25$  it reached  $x = 419.5$  instead of 377.3).
- Running the game: `game.py` ran for 6 seconds with no display and no errors. That only shows it doesn't crash. Please play it normally to confirm the motion looks smooth, and to record your "after" video.

## 07 USER PROMPT

Now implement Task 2 from the README: `enemy_tint(kind)`.

Requirements from the README:

- `enemy_tint(kind)` is called once per visible enemy per frame in draw.
- `kind` is "boss", "red", or "blue".
- It should return an (r, g, b) color override, or None to keep the default `ENEMY_COLORS` value.
- The purpose is to make later waves visually distinct.

Before coding:

1. Inspect the existing `enemy_tint()` function.
2. Inspect how the current wave number is stored and accessed.
3. Inspect the `draw()` method and existing `ENEMY_COLORS`.
4. Determine how the tint can be implemented without changing gameplay logic.

Then implement ONLY Task 2.

Use a simple, deterministic wave-based tinting scheme that:

- keeps the three enemy types visually distinguishable,
- makes later waves visually distinct,
- preserves the existing default colors when appropriate,
- does not change gameplay, movement, shooting, collision, scoring, or lives,
- does not introduce unnecessary refactoring.

After implementing it:

- Run/test the game.
- Verify that enemies are rendered correctly.
- Verify that the game still works normally.
- Tell me exactly what you changed and which files were modified.

Do NOT implement Task 3 or Task 4.

Do NOT commit anything. I will make the Git commit myself.

## 08 CLAUDE RESPONSE

Now testing: I'll check the tint values for each wave, render real frames to check enemy pixel colours, run gameplay through several wave clears with a fixed seed, and run the actual game.

## 09 CLAUDE RESPONSE

Let me look at two of the saved frames to check them visually.

## 10 CLAUDE RESPONSE

Task 2 is done: enemies now change colour by wave, and gameplay is unchanged. Only `game.py` was modified, nothing is committed, and Tasks 3 and 4 are still empty.

How the colours work

```
| Wave | Boss | Red rows | Blue row |
|---|---|---|---|
| 1, 4, 7, ??? | default green | default red | default blue |
| 2, 5, 8, ??? | gold | pink | teal |
| 3, 6, 9, ??? | amber | crimson | mint |
```

The three enemy types always have clearly different colours within a wave. None of the palettes use the purple a boss turns when hit once, so that warning still stands out.

What I changed in game.py

1. `enemy_tint(kind)`: it now looks up the colour for the current wave from a new `WAVE_PALETTES` list of three palettes, which repeats. Wave 1's entry is `None`, so the existing or `ENEMY_COLORS[...]` line in `draw()` keeps the default colours.
2. A new module variable `tint_wave`: `enemy_tint` only receives `kind`, and the wave number lives only in `Game.wave`. I added two lines at the top of `spawn_wave(wave)` so it saves the wave number for the tint. `spawn_wave` runs at game start, on R reset and at every new wave, so the colour always matches the current wave.

I didn't change `draw()`, the `enemy_tint` call, `on_wave_start` or any gameplay code. A global variable isn't the neatest design. I chose it because it keeps the README's `enemy_tint(kind)` signature exactly as specified.

Testing

- Colour per wave: waves 1??7 return the expected palette, repeating every three waves.
- Rendered frames: I drew settled formations for waves 1??4 and read the pixel colour of each enemy. All matched exactly, including the defaults on waves 1 and 4. A boss hit once still draws purple (170, 90, 220). The screenshots of waves 2 and 3 look right.
- Gameplay: a scripted 120-second run reached wave 9 with score 15,100 and no errors. The wave used for the colours matched the game's wave on every frame, and pressing R reset the colours to the wave-1 defaults.
- Real game: python game.py ran for 6 seconds with no display and didn't crash.

The mint "blue" enemies on wave 3 look similar to the wave-1 boss green. That doesn't matter within a wave, but if it bothers you, swap that one colour.

## 11 USER PROMPT

Before I commit Task 2, review the current git diff carefully.

Task 2 is ONLY the implementation of `enemy_tint(kind)`.

Requirements:

- Keep the `enemy_tint` implementation and the minimum changes required to make it work.
- Do NOT modify keyboard handling, controls, input mappings, game loop behavior, collision, movement, shooting, scoring, lives, or any unrelated functionality.
- Revert any unrelated changes that were introduced during testing, including the key-mapping/list-to-dict change you mentioned.
- Do not implement Task 3 or Task 4.
- Do not commit anything.

After cleanup:

1. Show me the final git diff for game.py.
2. List every logical change remaining in the diff.
3. Confirm that every remaining change is directly required for Task 2.
4. Run a basic test again without modifying unrelated code.

Do not make any additional feature changes.

## 12 CLAUDE RESPONSE

The diff only contains Task 2 and is ready for you to commit. Nothing needed reverting, and I haven't committed anything.

1. Final diff for game.py

"diff

```
@@ -14,9 +14,19 @@ def bezier(p0, p1, p2, p3, t):
     return pygame.Vector2(x, y)
```

```
+tint_wave = 1
+
+## Palettes cycle by wave; None keeps ENEMY_COLORS. Hues avoid the damaged-boss purple.
+WAVE_PALETTES = [
+    None,
+    {"boss": (240, 200, 60), "red": (240, 110, 170), "blue": (60, 200, 200)},
+    {"boss": (230, 140, 50), "red": (200, 50, 100), "blue": (120, 230, 140)},
+]
+
+
+def enemy_tint(kind):
```

```
"""Return an (r, g, b) colour override for an enemy kind, or None for the default."""
```

```
- pass
+ palette = WAVE_PALETTES[(tint_wave - 1) % len(WAVE_PALETTES)]
+ return palette[kind] if palette else None
```

```
def on_wave_start(wave):
    @@ -74,6 +84,8 @@ class Enemy:
```

```
def spawn_wave(wave):
    + global tint_wave
    + tint_wave = wave
    enemies, index = [], 0
    ,
```

game.py is the only modified file.

2. Every change left in the diff, and 3. why Task 2 needs it

```
| # | Change | Why Task 2 needs it |
|---|---|---|
| 1 | WAVE_PALETTES list: default colours, then two alternate palettes | Holds the colours that make
later waves look different. |
| 2 | enemy_tint(kind) body replaces pass | This is the Task 2 function. It returns None on waves 1,
4, 7??? so the default colours are kept. |
| 3 | tint_wave = 1 module variable | enemy_tint only receives kind, so it needs another way to know
the current wave. |
| 4 | Two lines in spawn_wave that set tint_wave | Updates that variable at game start, on reset and
at each new wave. It doesn't change how enemies are created. |
```

The diff doesn't touch keyboard handling, controls, the game loop, collision, movement, shooting, scoring, lives, on\_wave\_start (Task 3) or shield\_charges (Task 4).

4. Basic test, run again with no files changed

- enemy\_tint returns the default colours on waves 1 and 4, and the two alternate palettes on waves 2 and 3.
- python game.py' ran for 6 seconds with no display and didn't crash.

### 13 USER PROMPT

Now implement Task 3 from the README: on\_wave\_start(wave).

README requirements:

- on\_wave\_start(wave) is called once from spawn\_wave(wave), right after that wave's enemy list is built.
- It is called at game start and every time a wave is cleared.
- It receives the 1-indexed wave number.
- Its return value is ignored.
- The README suggests showing a "WAVE 3" banner for a second, or ramping up the dive rate.

Before coding:

1. Inspect the existing on\_wave\_start(wave) function.
2. Inspect spawn\_wave(wave) and determine exactly where on\_wave\_start is called.
3. Inspect the existing game drawing and update loop.
4. Inspect whether the game already has any text/banner/timer mechanism that can be reused.

Implement ONLY Task 3.

Use the README's suggested "WAVE X" banner approach:

- Display a visible "WAVE X" banner whenever a new wave starts.
- The banner should remain visible for approximately one second.
- It must work for wave 1 and every subsequent wave.
- Do NOT use time.sleep() or block the game loop.
- Use the existing Pygame architecture and game loop.
- Keep the implementation simple and readable.
- Do not change enemy movement, shooting, collision, scoring, lives, controls, or other gameplay.
- Do not modify Task 2's enemy tint functionality.
- Do not implement Task 4.

After implementation:

1. Run/test the game normally.
2. Verify the banner appears at the start of a wave.
3. Verify it disappears automatically after approximately one second.
4. Verify the game continues normally while the banner is displayed.
5. Inspect the final git diff and make sure only Task 3 changes are present.

6. Tell me exactly what files and code you changed.

Do NOT commit anything. I will make the Git commit myself.

## 14 CLAUDE RESPONSE

Nine enemies is right: each enemy starts 0.12 s after the previous one, so about 9 begin flying in the first second. Checking the screenshot and the diff:

## 15 CLAUDE RESPONSE

Task 3 works: a "WAVE X" banner appears in the centre of the screen at the start of every wave and disappears after about one second. The game keeps running while it shows. Only game.py changed, and nothing is committed.

What I changed in game.py

1. Two new module variables and a constant, above `on_wave_start`: `banner_text`, `banner_until` and `BANNER_MS = 1000`. `on_wave_start` only receives the wave number, and its return value is ignored, so these are how it tells `draw()` what to show and until when.
2. `on_wave_start(wave)`: replaces pass. It sets the text to "WAVE {wave}" and the expiry to the current time plus 1000 ms, using `pygame.time.get_ticks()`.
3. `Game.draw()`: three lines added after the score line. While the expiry time hasn't passed, it draws the banner centred on the screen with the existing font and the same yellow as "GAME OVER".

No `time.sleep()` and no change to `update()` or the main loop; the banner disappears once the time passes. I didn't change `spawn_wave` (it already calls the hook at the right place), the tint code, controls, movement, shooting, collision, scoring, lives or `shield_charges`.

The diff: it contains only the Task 3 changes above. Task 2 was already committed (b6180d2), so it doesn't appear.

Testing (real time, at 60 fps)

```
| Case | Banner text | Visible for | Then gone |
|---|---|---|---|
| Game start | WAVE 1 | ~994 ms | yes |
| After wave 1 cleared | WAVE 2 | ~999 ms | yes |
| After wave 2 cleared | WAVE 3 | ~1003 ms | yes |
| R reset | WAVE 1 | ~989 ms | yes |
```

- Game keeps running: while the banner was up, game time advanced and enemies flew in. About 9 had started moving by the end of the second, which matches the 0.12 s gap between enemy entries.
- Screenshot: the wave-1 frame shows the banner centred while the first enemies fly in. The wave-1 default colours still apply, so Task 2 is unaffected.
- Real game: `python game.py` ran for 6 seconds with no display and didn't crash.

Two things to know:

- The banner uses the same font size as the score line. You asked for a simple version, so I didn't add a larger font. It's easy to add if you want it bigger.
- The banner times itself with real time, not game time. That would only matter if the game ever paused, which it currently can't.

## 16 USER PROMPT

Now implement Task 4 from the README: `shield_charges(wave)`.

README requirements:

- `shield_charges(wave)` is called at game start and at the start of every new wave.
- It receives the current wave number.
- Its result is assigned as:  
`self.shield = shield_charges(self.wave)` or 0
- It should return an integer number of hits the player can absorb before losing a ship.
- Returning None or 0 disables the shield.
- The existing `hit_player()` logic that consumes shield charges is already implemented.
- The README suggests granting 1 shield charge every 3 waves.

Before coding:

1. Inspect the existing `shield_charges(wave)` function.
2. Inspect how `self.shield` is initialized.
3. Inspect `hit_player()` to understand how shield charges are consumed.
4. Inspect `spawn_wave()` to verify when `shield_charges()` is called.

Implement ONLY Task 4.

Use the README's suggested behavior:

- Grant 1 shield charge every 3 waves.

- Otherwise return 0 or None.
- Make sure the behavior works correctly for wave 1, 2, 3, 4, 5, 6, etc.
- Do not rewrite the existing shield consumption logic.
- Do not change unrelated gameplay.
- Do not modify Task 1, Task 2, or Task 3.
- Do not implement any other features.

After implementation:

1. Run/test the game.
2. Verify that shield charges are granted on the appropriate waves.
3. Verify that shield charges are consumed correctly when the player is hit.
4. Inspect the final git diff.
5. Make sure ONLY Task 4 changes are present.
6. Tell me exactly what files and code you changed.

Do NOT commit anything. I will make the Git commit myself.

## 17 CLAUDE RESPONSE

Task 4 is done: the player gets 1 shield charge on waves 3, 6, 9 and so on, and no shield on other waves. It's a one-line change in game.py, and nothing is committed.

The change

“diff

def shield\_charges(wave):

"""Return how many hits the player's shield can absorb this wave, or None to disable the shield."""

- pass

+ return 1 if wave % 3 == 0 else 0

,

This is the only change in the diff. Tasks 1??3 are already committed (latest 56a2e1f) and untouched. I didn't change hit\_player() or anything else.

One correction to the brief: shield\_charges() isn't called from spawn\_wave(). It's called from Game.reset() (game.py:116) and Game.update() (game.py:174), each right after spawn\_wave(). The timing is the same, so it works as the README intends. Because the value is reassigned each wave, an unused charge doesn't carry over to the next wave.

Testing

- Charges per wave:

|                |     |     |     |     |     |     |     |     |     |   |
|----------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|---|
| Wave           | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   |   |
| ---            | --- | --- | --- | --- | --- | --- | --- | --- | --- |   |
| Shield charges | 0   | 0   | 1   | 0   | 1   | 0   | 1   | 0   | 0   | 1 |

The game's own shield value matched this table for each wave when I cleared waves 1??9 through the normal game code.

- Charges are used up correctly on wave 3:
  - The first hit uses the charge, keeps the ship and life, and gives 1 s of invulnerability.
  - A hit during that second does nothing.
  - The next hit costs a life as normal (3 ??' 2).
  - A real enemy shot hitting the ship during normal play also used the charge instead of a life.
- No shield on wave 1: a hit costs a life as before.
- No carry-over: an unused wave-3 charge resets to 0 on wave 4.
- Reset: R brings back wave 1 with no shield, the default colours and the "WAVE 1" banner, so Tasks 2 and 3 still work.
- Real game: python game.py' ran for 6 seconds with no display and didn't crash.

The screen doesn't show how many shield charges you have, so in your "after" video the shield will only show up as a hit that doesn't cost a ship, followed by a short blink. Adding the count to the score line would be a one-line change if you want it, but it's beyond what Task 4 asks for.